
EEEM071 Advanced Topics in Computer Vision and Deep Learning

Lab Week 6

Introduction to NumPy and PyTorch

Dr. Xiatian Zhu

xiatian.zhu@surrey.ac.uk

1 Introduction

Prerequisites: A basic understanding of the Python Programming Language (Week 5 - Lab sheet!)

The objective of today's lab is to introduce you to the NumPy and PyTorch libraries.

“**Numerical Python**” (**NumPy**) and **PyTorch** are popular scientific computing libraries in Python that provide support for arrays of numerical data, specifically multidimensional arrays. Although both NumPy and PyTorch are written in Python, their underlying numerical operations are often implemented in C or C++ to provide more efficient and convenient ways to perform operations on arrays.

NumPy is a fundamental package for scientific computing in Python. It provides support for multi-dimensional arrays and matrices, as well as functions that operate on them. It is commonly used to preprocess and manipulate data before feeding it into machine learning models. For example, you can use NumPy to reshape, normalize, or scale datasets, as well as to split datasets into training and testing sets.

PyTorch is an open-source machine learning library developed by Meta. It is designed to provide flexibility and speed for building deep learning models. PyTorch provides a range of functions and tools for building and training neural networks, including automatic differentiation, GPU acceleration, and distributed training.

Both NumPy and PyTorch can be used to create, manipulate, and transform numerical data in ways that are essential to building and training machine learning models. They are key tools in the modern data scientist's toolkit.

Happy Programming! Please open this [colab](#) notebook for NumPy exercise and this [colab](#) notebook for PyTorch exercise. Follow the instruction in the lab sheet week 5-10 to learn how to copy to your google drive.

2 Arrays

An array is a data structure that holds a *fixed-size* sequential collection of elements of the same type. It allows us to store and access multiple values using a single variable name (like a container!). Note that NumPy arrays are *homogeneous*, meaning that all the elements in the array must be of *the same data type*.

Instruction: Run the code in the section **Arrays** of the colab notebook.

2.1 Datatypes

Data types help define the type of data being used in a program, and how that data is stored and processed in memory. Choosing the right data type can have a significant impact on the performance and accuracy of your model. For example, using a higher precision data type like `double` can improve the accuracy of the model, but at the cost of increased memory usage and slower computation time.

NumPy provides a range of data types to work with, each with its own specific use cases. Here are some of the most commonly used data types in NumPy:

Integer data types:

- `int8`, `int16`, `int32`, `int64`: These are signed integer data types with sizes of 8, 16, 32, and 64 bits respectively. They are useful for representing whole numbers within a specific range, and are commonly used in a variety of applications such as image processing, audio signal processing, and machine learning.
- `uint8`, `uint16`, `uint32`, `uint64`: These are unsigned integer data types with sizes of 8, 16, 32, and 64 bits respectively. They are useful for representing values that can only be positive, such as image data or color channels.

Float data types:

- `float16`, `float32`, `float64`: These are floating-point data types with sizes of 16, 32, and 64 bits respectively. They are used for representing decimal numbers and are commonly used in scientific computing and machine learning applications.
- `complex64`, `complex128`: These are complex number data types that consist of two floating-point numbers of 32 or 64 bits respectively. They are used for representing complex numbers, which are commonly encountered in scientific and engineering applications.

Boolean data types:

- `bool`: This is a Boolean data type that represents the values `True` or `False`. It is commonly used in conditional statements and logical operations.

String data types:

- `string`: This is a fixed-length string data type. It is useful for representing text data in applications such as natural language processing.
- `unicode`: This is a fixed-length string data type that is used for representing text data that includes non-ASCII characters, such as in internationalization and localization applications.

Instruction: Run the code in the section **Datatypes** of the colab notebook.

Choosing the appropriate data type for a given task can be crucial for achieving optimal performance and accuracy in NumPy applications. It can also help prevent errors and improve code readability.

Similarly, the common data types in PyTorch including but not limited to:

1. Float (`torch.float32`): This is the default data type in PyTorch and is commonly used for representing input data and model parameters.
2. Double (`torch.float64`): This data type is used when high precision is required, such as in scientific computing.

3. Half (`torch.float16`): This data type is used for models with low memory requirements, such as those used in mobile devices or embedded systems.
4. Integer (`torch.int32`, `torch.int64`): These data types are used for representing discrete values, such as class labels or indices.
5. Boolean (`torch.bool`): This data type is used for representing True or False values.

In summary, understanding data types in NumPy/PyTorch is critical for developing and training deep learning models that are accurate, efficient, and optimized for specific hardware and memory requirements.

2.2 Array Broadcasting

Array broadcasting is a technique in numpy and other similar libraries that allows you to perform operations between arrays that *may not have the same shape*. Essentially, broadcasting lets you apply an operation to two or more arrays that may have different dimensions or shapes, and numpy will automatically "stretch" the smaller array to fit the larger one. This way, you can perform operations on arrays that would otherwise be impossible or inefficient to do.

For example, let's say you have two arrays: A, which has the shape (3, 2)¹, and B, which has the shape (2,)². Normally, you wouldn't be able to perform an element-wise addition between these arrays because their shapes don't match. However, with broadcasting, you can simply add the arrays together, and numpy will automatically "broadcast" the smaller array B to match the shape of A, by replicating its elements along the first dimension. This way, you end up with an array of shape (3, 2) that represents the element-wise sum of A and B.

The broadcasting techniques in NumPy:

- Broadcasting a scalar value to an array.
- Broadcasting a one-dimensional array to a two-dimensional array.
- Broadcasting two arrays with different shapes.

Instruction: Run the code in the section **Array Broadcasting** of the colab notebook.

The benefit of array broadcasting is that it allows you to perform mathematical operations on arrays of different shapes and sizes without having to create multiple copies of the same data. This can help to save memory and improve performance when working with large arrays in scientific computing or machine learning applications.

2.3 Shape Manipulation

Shape manipulation refers to changing the shape or size of an array in NumPy or PyTorch without changing the underlying data. It is useful when you need to modify the shape of your data to fit a certain computation or operation.

Here are some of the shape manipulation functions available in NumPy:

1. Reshaping arrays: You can reshape an array to have a different shape using the reshape method. `reshape()`: This function is used to change the shape of a numpy array. It takes a new shape as an argument and returns a new array with the same data but with a new shape.

¹It means 3 rows and 2 columns.

²It means 2 rows only.

2. Transposing arrays: You can transpose an array (i.e., switch its rows and columns) using the `transpose` method or the `T` attribute.
`resize()`: This function is similar to `reshape()`, but it changes the shape of the original array in-place.
3. Flattening arrays: You can flatten a multi-dimensional array into a 1D array using the `flatten` method.
`flatten()`: This function returns a 1-D numpy array that contains all the elements of the original array.
4. Expanding dimensions: You can add dimensions to an array using the `newaxis` keyword. For example, you can add a new dimension to a 1D array to create a 2D array with one row and multiple columns.
`np.newaxis`: is used to increase the dimension of the existing array by one more dimension. It is used to create a new axis or add a new dimension to an existing NumPy array.

Instruction: Run the code in the section **Shape Manipulation** of the colab notebook.

2.4 Conclusion

In summary, the main differences between Python's built-in lists and NumPy/PyTorch arrays are:

1. Data type: Python lists can hold elements of any data type, whereas NumPy and PyTorch arrays are designed to hold elements of a single data type (e.g., int, float, etc.). This makes them more memory-efficient and faster to process.
2. Performance: NumPy and PyTorch arrays are optimized for numerical computations and are much faster than Python lists for operations like element-wise arithmetic, matrix multiplication, and broadcasting.
3. Functionality: NumPy and PyTorch provide many built-in functions and methods for manipulating arrays, such as reshaping, slicing, indexing, and transposing, which are not available in Python lists.
4. Broadcasting: NumPy and PyTorch provide support for broadcasting, which allows for efficient element-wise operations between arrays of different shapes and sizes.

3 Introduction to PyTorch

Please open this [colab](#) notebook for PyTorch exercise.

3.1 Tensors

In PyTorch, a tensor is a multi-dimensional array that is similar to a NumPy array. However, PyTorch tensors can also be used on a GPU to accelerate computation, making it ideal for deep learning applications.

Tensors in PyTorch are created using the `torch.Tensor` class and can have any number of dimensions (rank) and different data types (e.g., float, int). Similar to NumPy, tensors can be manipulated using a variety of operations such as addition, subtraction, multiplication, and division.

In deep learning, tensors are used to store and manipulate the data used to train and evaluate neural networks. This includes the input data, the model parameters, and the output predictions. Since deep learning involves large amounts of data and complex computations, using PyTorch tensors can help make the training process more efficient and scalable.

Instruction: Run the code in the section **Introduction to Tensors** of the colab notebook.

3.2 Random Tensors

A random tensor is a tensor whose values are randomly generated by a random number generator. PyTorch provides several functions to create random tensors with different distributions.

For example, `torch.rand()` creates a tensor with values uniformly distributed between 0 and 1, while `torch.randn()` creates a tensor with values sampled from a standard normal distribution (mean=0, variance=1). These random tensors are useful in initializing the weights of neural networks and for generating random data for testing and experimentation.

Instruction: Run the code in the section **Random Tensors** of the colab notebook.

3.3 More Exercises

The following sections in the Colab notebook are designed to help you better understand PyTorch tensors and how to work with them effectively:

1. **Zeros and ones:** Creating tensors of all zeros or all ones is a common operation that can be done easily using PyTorch. It's a helpful tool when initializing tensors before filling them with real data.
2. **Creating a range of tensors and tensors-like:** PyTorch provides several functions for creating tensors or tensor-like objects, including `torch.arange()` and `torch.zeros_like()`. These functions allow you to easily create tensors with specific ranges or spacings.
3. **Tensor datatypes:** PyTorch tensors come in a variety of data types, each with its own range of values and precision levels. It's important to understand these data types when working with tensors to avoid errors. You can also refer to Section 2.1 to understand what is datatype.
4. **Getting information from tensors (tensor attributes):** You can access useful information about a tensor's shape, size, data type, and more using tensor attributes.
5. **Matrix multiplication:** Matrix multiplication is a common operation in linear algebra and machine learning. PyTorch provides efficient functions for performing matrix multiplication on tensors.
6. **Finding the min, max, mean, sum, etc (tensor aggregation):** PyTorch provides many functions for aggregating tensor data, such as finding the minimum, maximum, mean, and sum. These functions are useful for summarizing your data.
7. **Finding the positional min and max, the famous `argmin()` and `argmax()`:** In addition to finding the actual minimum and maximum values, PyTorch can also find the *position* of the minimum and maximum values within a tensor.
8. **Reshaping, stacking, squeezing and unsqueezing tensors:** You can change the shape of a tensor using functions like `reshape()`, `stack()`, `squeeze()`, and `unsqueeze()`. These functions are useful for adjusting the dimensions of your data.
9. **Indexing (selecting data from tensors):** You can select specific values from a tensor using indexing. PyTorch also provides advanced indexing options, such as selecting values based on specific conditions.
10. **PyTorch tensors & NumPy:** PyTorch integrates seamlessly with NumPy. This makes it easy to work with both PyTorch tensors and NumPy arrays in the same code.

11. **Reproducibility (trying to take random out of random):** Randomness is an important part of many machine learning algorithms, but it can make results difficult to reproduce. PyTorch provides tools for controlling the randomness of your code to make results more reproducible.

3.4 Gradient Descent from scratch

In machine learning, a model learns to make predictions based on a set of inputs (also known as features) and their corresponding outputs (also known as labels) by minimizing a loss function. The loss function measures how well the model's predictions match the true outputs. Gradient descent is an optimization algorithm used to find the set of parameters that minimizes the loss function.

The gradient is the vector of partial derivatives of the loss function with respect to each parameter in the model. It tells us the direction and magnitude of the steepest increase in the loss function. By taking steps in the opposite direction of the gradient, we can iteratively update the model's parameters to find the set that minimizes the loss function.

The loss function, as mentioned before, measures how well the model's predictions match the true outputs. It is a function that takes the model's predictions and the true outputs as inputs and returns a scalar value that indicates how far off the predictions are from the true outputs. The goal of training a machine learning model is to find the set of parameters that minimize the loss function, which means that the model's predictions are as close as possible to the true outputs.

Gradient descent is used in machine learning to update the weights or parameters of a model during training. In gradient descent, we start with an initial guess for the values of our parameters, and then we iteratively adjust these values by moving in the direction of steepest descent, which is the direction in which the function is decreasing the fastest. This is done by calculating the gradient of the function at the current point, which gives us the direction in which the function is increasing, and then moving in the opposite direction to go downhill towards the minimum.

The process of updating the weights continues until the loss function converges to a minimum, indicating that the model has learned the underlying patterns in the data.

Instruction: Run the code in the section **Gradient descent from scratch** of the colab notebook.

In PyTorch, `requires_grad_()` is a method that can be called on a tensor to set the `requires_grad` attribute of that tensor.

When `requires_grad` is set to `True`, PyTorch tracks all the operations performed on that tensor and creates a computational graph. This graph is used to calculate gradients during backpropagation.

The reason why `requires_grad_()` is used is to enable automatic differentiation, which is a key feature of PyTorch that makes it easier to implement machine learning algorithms. By setting `requires_grad` to `True`, PyTorch can automatically calculate gradients with respect to that tensor and update the tensor's values during training.